

Medios

Damian Nicolalde R.

danicolalde@puce.edu.ec

Vídeo

- Los vídeos son un método extremadamente eficaz de comunicación. Nadie puede negar la importancia de los vídeos en los sitios web y aplicaciones de hoy en día, y mucho menos aquellos que se encargan de desarrollar las tecnologías para la Web
- Esta es la razón por la que HTML5 incluye un elemento con el único propósito de cargar y reproducir vídeos

<video>—Este elemento inserta un vídeo en el documento.

El elemento **<video>** requiere etiquetas de apertura y cierre, y solo algunos parámetros para cumplir su función

La sintaxis es sencilla y solo es obligatorio el atributo src

Atributos para declarar el área que ocupa el vídeo y configurar el reproductor

- **src**—Este atributo especifica la URL del vídeo a reproducir.
- **width**—Este atributo determina el ancho del área del reproductor.
- **height**—Este atributo determina la altura del área del reproductor.
- **controls**—Este es un atributo booleano. Si está presente, el navegador muestra una interfaz para permitir al usuario controlar el vídeo.
- **autoplay**—Este es un atributo booleano. Si está presente, el navegador reproduce el vídeo automáticamente tan pronto como puede.
- **loop**—Este es un atributo booleano. Si está presente, el navegador reproduce el vídeo una y otra vez.
- **muted**—Este es un atributo booleano. Si está presente, el audio se silencia.
- **poster**—Este atributo especifica la URL de la imagen que se mostrará mientras el navegador espera que el vídeo se reproduzca.
- **preload**—Este atributo determina si el navegador debería comenzar a cargar el vídeo antes de ser reproducido. Acepta tres valores: **none**, **metadata** o **auto**. El primer valor indica que el vídeo no se debería cargar y generalmente se utiliza para minimizar tráfico web. El segundo valor, **metadata**, recomienda al navegador descargar información acerca del recurso, como las dimensiones, la duración, el primer cuadro, etc. El tercer valor, **auto**, solicita al navegador que descargue el archivo tan pronto como sea posible (este es el valor por defecto).

Cargando un vídeo

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>Reproductor de Video</title>
6  </head>
7  <body>
8      <section>
9          <video src="trailer.mp4" controls autoplay>
10             </video>
11      </section>
12 </body>
13 </html>
```

- El elemento **<video>** carga el vídeo especificado por el atributo **src** y reserva un área del tamaño del vídeo en el documento, pero el vídeo se reproduce.
- Tenemos que indicarle al navegador cuándo queremos que reproduzca el vídeo o facilitar las herramientas para dejar que el usuario decida.
- Existen dos atributos que podemos agregar al elemento para este propósito: **controls** y **autoplay**.
- El atributo **controls** indica al navegador que debería incluir sus propios elementos (botones y barras) para permitir al usuario controlar el vídeo,
- El atributo **autoplay** solicita al navegador que comience a reproducir el vídeo tan pronto como pueda.

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="utf-8">
5   <title>Reproductor de Video</title>
6 </head>
7 <body>
8   <section>
9     <video src="trailer.mp4" width="720" height="400" controls>
10    </video>
11  </section>
12 </body>
13 </html>
```

- Por defecto, los navegadores determinan el tamaño de área del vídeo a partir de su tamaño original, pero podemos definir un tamaño personalizado con los atributos **width** y **height**.
- Estos atributos son como los atributos del elemento ****, declaran las dimensiones del elemento en píxeles. Cuando están presentes, el tamaño del vídeo se ajusta para adaptarlo a estas dimensiones, pero no tienen el propósito de comprimir o expandir el vídeo.
- Podemos usarlos para limitar el área ocupada por el medio y preservar la coherencia en nuestro diseño.

Incluyendo una imagen que represente el vídeo

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <title>Reproductor de Video</title>
</head>
<body>
  <section>
    <video src="trailer.mp4" width="720" height="400" preload controls
    loop poster="poster.jpg">
  </video>
  </section>
</body>
</html>
```

Este ejemplo carga el vídeo tan pronto como se carga el documento, reproduce el vídeo continuamente y muestra una imagen en lugar del vídeo mientras este no se reproduce.

La Figura, muestra lo que vemos antes de pulsar el botón para iniciar la reproducción.



Formatos de vídeo

- A pesar de que el elemento **<video>** y sus atributos son estándar, no existe un formato estándar para la web, algunos de los navegadores admiten un grupo de codificadores y otros no, y el codificador que se usa en el formato MP4 (el único que admiten navegadores destacados como Safari e Internet Explorer) se distribuye bajo licencia comercial.
- Las opciones más comunes actualmente son OGG, MP4, y WebM. Estos formatos son contenedores de vídeo y audio.
 - OGG contiene codificadores de vídeo Theora y audio Vorbis
 - MP4 contiene H.264 para vídeo y AAC para audio, y
 - WebM usa VP8 para vídeo Vorbis para audio.
- Actualmente, OGG y WebM son compatibles con Mozilla Firefox, Google Chrome y Opera, mientras que MP4 funciona en Safari, Internet Explorer y Google Chrome. HTML contempla este escenario e incluye un elemento adicional que funciona con el elemento **<video>** para definir las posibles fuentes del vídeo.

<source>

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4  |   <meta charset="utf-8">
5  |   <title>Reproductor de Video</title>
6  </head>
7  <body>
8  |   <section>
9  |   <video width="720" height="400" controls>
10 |       <source src="trailer.mp4">
11 |       <source src="trailer.ogg">
12 |   </video>
13 |   </section>
14 </body>
15 </html>
```

- Este elemento define una fuente para un vídeo. Debe incluir el atributo src para indicar la URL del archivo
- Cuando necesitamos especificar múltiples fuentes para el mismo vídeo, tenemos que reemplazar el atributo src en el elemento <video> por elementos <source> entre las etiquetas, como en el siguiente ejemplo.

IMPORTANTE

- Los navegadores requieren que los vídeos se envíen en el servidor con los correspondientes tipos MIME. Cada archivo tiene un tipo MIME asociado para indicar el formato de su contenido.
 - Por ejemplo, el tipo MIME para un archivo HTML es text/html.
- Los servidores ya están configurados para la mayoría de los formatos de vídeo, pero normalmente no para nuevos formatos como OGG o WEBM.
- La forma de incluir este nuevo tipo MIME depende del tipo de servidor. Una manera sencilla de hacerlo es agregar una nueva línea al archivo .htaccess. La mayoría de los servidores incluyen este archivo de configuración en el directorio raíz de todo sitio web.
- La sintaxis correspondiente es Addtype MIME/type extension (por ejemplo, AddType video/ogg ogg).

Audio

- El audio es un medio que no tiene la misma popularidad en la Web que los vídeos. Podemos filmar un vídeo con una cámara personal que será visto por millones de personas, pero lograr el mismo resultado con un archivo de audio sería casi imposible. Sin embargo, el audio aún se encuentra presente en la Web a través de shows de radio y podcasts. Por esta razón, HTML5 también ofrece un elemento para reproducir archivos de audio.

<audio>—Este elemento inserta audio en el documento.

- Este elemento trabaja de la misma forma y comparte varios atributos con el elemento <video>

Atributos de <audio>

- **src**—Este atributo especifica la URL del archivo a reproducir.
- **controls**—Este es un atributo booleano. Si está presente, activa la interfaz que facilita el navegador por defecto.
- **autoplay**—Este es un atributo booleano. Si está presente, el navegador reproduce el audio automáticamente tan pronto como puede.
- **loop**—Este es un atributo booleano. Si está presente, el navegador reproduce el audio una y otra vez.
- **preload**—Este atributo determina si el navegador debe comenzar a cargar el archivo de audio antes de reproducirse. Acepta tres valores: **none**, **metadata** o **auto**. El primer valor indica que el audio no se debería cargar y generalmente se utiliza para minimizar tráfico web. El segundo valor, **metadata**, recomienda al navegador que descargue información acerca del recurso, como la duración del audio. El tercer valor, **auto**, solicita al navegador que descargue el archivo tan pronto como le sea posible (este es el valor por defecto).

La implementación del elemento **<audio>** en nuestro documento es muy similar a la del elemento **<video>**. Solo tenemos que especificar la fuente y ofrecer al usuario la posibilidad de iniciar la reproducción.

Reproduciendo audio con el elemento <audio>

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4  |   <meta charset="utf-8">
5  |   <title>Reproductor de Audio</title>
6  </head>
7  <body>
8  |   <section>
9  | |   <audio src="cancion.mp3" controls>
10 | |   </audio>
11 |   </section>
12 </body>
13 </html>
```

- El código HTML del ejemplo debería ser suficiente para reproducir un sonido en la Web, pero no lo es.
- MP3 se encuentra bajo licencia comercial, por lo que no es compatible con navegadores como Mozilla Firefox u Opera.
- Vorbis (el codificador de audio del contenedor OGG) es compatible con estos navegadores, pero no con Safari e Internet Explorer. Por lo tanto, una vez más, tenemos que usar el elemento **<source>** para facilitar al menos dos formatos para que el navegador pueda elegir.

<source>

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4  |   <meta charset="utf-8">
5  |   <title>Reproductor de Audio</title>
6  </head>
7  <body>
8  |   <section>
9  |   |   <audio id="medio" controls>
10 |   |   |   <source src="cancion.mp3">
11 |   |   |   <source src="cancion.ogg">
12 |   |   </audio>
13 |   </section>
14 </body>
15 </html>
```

- Este ejemplo reproduce una canción en todos los navegadores con controles por defecto.
- Aquellos que no pueden reproducir MP3 usarán el archivo OGG, y viceversa. Solo tenemos que recordar que MP3, así como el formato MP4 para vídeo, se distribuyen bajo licencias comerciales y solo podemos emplearlos bajo las circunstancias permitidas por sus licencias

API Media

- Si agregamos el atributo **controls** a los elementos **<video>** y **<audio>** activamos la interfaz por defecto que facilita cada navegador.
- Esta interfaz puede resultar útil en sitios web sencillos o pequeñas aplicaciones, pero en un ambiente profesional, donde cada detalle cuenta, es necesario disponer de un control absoluto sobre todo el proceso y facilitar un diseño coherente para todos los dispositivos y aplicaciones.
- Los navegadores incluyen la API Media para ofrecer una alternativa personalizada. Esta API es un grupo de propiedades, métodos y eventos diseñados para manipular e integrar vídeo y audio en nuestro documentos.
- Combinando esta API con HTML y CSS podemos crear nuestros propios reproductores de vídeo o audio con los controles que queramos
- La API se incluye en los objetos **Element** que representan los elementos **<video>** y **<audio>**

Propiedades

- **paused**—Esta propiedad devuelve **true** si el medio se ha pausado o aún no ha comenzado a reproducirse.
- **ended**—Esta propiedad devuelve **true** si el medio ha terminado de reproducirse.
- **duration**—Esta propiedad devuelve la duración del medio en segundos.
- **currentTime**—Esta propiedad declara o devuelve un valor que determina la posición en la que el medio se está reproduciendo o debería comenzar a reproducirse.
- **volume**—Esta propiedad declara o devuelve el volumen del medio. Acepta valores entre 0.0 y 1.0.
- **muted**—Esta propiedad declara o devuelve el estado del audio. Los valores son **true** (silenciado) o **false** (no silenciado).
- **error**—Esta propiedad devuelve el valor del error ocurrido.

Propiedades

- **buffered**—Esta propiedad ofrece información sobre las partes del archivo que ya se han descargado. Como el usuario puede forzar al navegador a descargar el medio desde diferentes posiciones en la línea de tiempo, la información que devuelve **buffered** es un array que contiene cada parte del medio que se ha descargado, no solo la que comienza desde el inicio del medio. A los elementos del array se puede acceder mediante los métodos **end()** y **start()**. Por ejemplo, el código **buffered.start(0)** devuelve el tiempo en el que comienza la primera parte del medio y **buffered.end(0)** devuelve el tiempo en el que esa misma porción termina.

Métodos para manipular el medio.

- **play()**—Este método reproduce el medio.
- **pause()**—Este método pausa el medio.
- **load()**—Este método carga el archivo del medio. Es útil cuando necesitamos cargar el medio de antemano en aplicaciones dinámicas.
- **canPlayType(tipo)**—Este método devuelve un valor que determina si un formato de archivo es compatible con el navegador o no. El atributo **tipo** es un tipo MIME que representa el formato del medio, como video/mp4 o video/ogg. El método puede devolver tres valores dependiendo de lo seguro que esté de que puede reproducir el medio: una cadena de caracteres vacía (el formato no es compatible), el texto "maybe" (a lo mejor) y el texto "probably" (probablemente).

Esta API incluye elementos para informar sobre la situación actual del medio, como el progreso al descargar el archivo, si el vídeo ha llegado al final, o si se ha pausado o se está reproduciendo, entre otros. Los siguientes son los más usados:

- **progress**—Este evento se desencadena periódicamente para ofrecer una actualización del progreso de la descarga del medio. A la información se puede acceder a través del atributo **buffered**.
- **canplaythrough**—Este evento se desencadena cuando el medio completo se puede reproducir sin interrupción. El estado se establece considerando la velocidad de descarga actual y asumiendo que seguirá siendo la misma durante el resto del proceso. Existe otro evento para este propósito llamado **canplay**, pero no considera toda la situación y se desencadena cuando apenas hay disponibles unos pocos cuadros.
- **ended**—Este evento se desencadena cuando el medio termina de reproducirse.
- **pause**—Este evento se desencadena cuando se pausa el medio.
- **play**—Este evento se desencadena cuando el medio comienza a reproducirse.
- **error**—Este evento se desencadena cuando ocurre un error. Se despacha al elemento **<source>** correspondiente con la fuente del medio que ha producido el error.

Reproductor de vídeo

- Todo reproductor de vídeo necesita un panel de control con algunas herramientas básicas.
- La API no facilita una manera de crear estos botones o barras, tenemos que definirlas nosotros usando HTML y CSS.
- El siguiente ejemplo crea una interfaz con dos botones para reproducir y silenciar el vídeo, un elemento `<div>` para representar una barra de progreso y un deslizador para controlar el volumen.

```

1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>Reproductor de Video</title>
6      <link rel="stylesheet" href="reproductor.css">
7      <script src="reproductor.js"></script>
8  </head>
9  <body>
10     <section id="reproductor">
11         <video id="medio" width="720" height="400">
12             <source src="Post-PC.mp4">
13             <source src="Post-PC.ogg">
14         </video>
15         <nav>
16             <div id="botones">
17                 <input type="button" id="reproducir" value=">">
18                 <input type="button" id="silenciar" value="Silencio">
19             </div>
20             <div id="barra">
21                 <div id="progreso"></div>
22             </div>
23             <div id="control">
24                 <input type="range" id="volumen" min="0" max="1" step="0.1"
25                     value="0.6">
26             </div>
27             <div class="recuperar"></div>
28         </nav>
29     </section>
30 </body>
31 </html>

```

```

1  #reproductor {
2      width: 720px;
3      margin: 20px auto;
4      padding: 10px 5px 5px 5px;
5      background: #999999;
6      border: 1px solid #666666;
7      border-radius: 10px;
8  }
9  #reproducir, #silenciar {
10     padding: 5px 10px;
11     width: 70px;
12     border: 1px solid #000000;
13     background: #DDDDDD;
14     font-weight: bold;
15     border-radius: 10px;
16 }
17 nav {
18     margin: 5px 0px;
19 }
20 #botones {
21     float: left;
22     width: 145px;
23     height: 20px;
24     padding-left: 5px;
25 }
26 #barra {
27     float: left;
28     width: 400px;
29     height: 16px;
30     padding: 2px;
31     margin: 2px 5px;
32     border: 1px solid #CCCCC;
33     background: #EEEEEE;
34 }
35 #progreso {
36     width: 0px;
37     height: 16px;
38     background: rgba(0,0,150,.2);
39 }
40 .recuperar {
41     clear: both;
42 }

```



Play

Barra

Volumen

reproductor.js

```
1  var maximo, mmedio, reproducir, barra, progreso, silenciar, volumen,
2  bucle;
3  function iniciar() {
4      maximo = 400;
5      mmedio = document.getElementById("medio");
6      reproducir = document.getElementById("reproducir");
7      barra = document.getElementById("barra");
8      progreso = document.getElementById("progreso");
9      silenciar = document.getElementById("silenciar");
10     volumen = document.getElementById("volumen");
11     reproducir.addEventListener("click", presionar);
12     silenciar.addEventListener("click", sonido);
13     barra.addEventListener("click", mover);
14     volumen.addEventListener("change", nivel);
15 }
```

- En la función **iniciar()** se crea una referencia a cada elemento y también inicializa la variable **maximo** para establecer el tamaño máximo de la barra de progreso (400 píxeles).
- Además, declara listeners para múltiples eventos que nos permiten responder a las acciones del usuario.
- Hay varias acciones a las que tenemos que prestar atención: cuando el usuario hace clic en los botones ">" (reproducir) y Silencio, cuando cambia el volumen desde el elemento **volumen** o cuando hace clic en la barra de progreso para retroceder o adelantar el vídeo.
- Agregamos listeners para el evento click a los elementos reproducir, silenciar, y barra, y uno para el evento change al elemento volumen para controlar el volumen.

Cada vez que el usuario hace clic en uno de estos elementos o mueve el deslizador, se ejecutan las funciones correspondientes:

`presionar()` para el botón ">" (reproducir), `sonido()` para el botón Silencio, `mover()` para la barra de progreso y `nivel()` para el deslizador.

La primera función que tenemos que implementar es `presionar()`. Esta función se encarga de reproducir o pausar el vídeo cuando se pulsan los botones ">" (reproducir) o "| |" (pausar).

reproductor.js

```
16 function presionar() {  
17     if (!medio.paused && !medio.ended) {  
18         medio.pause();  
19         reproducir.value = ">";  
20         clearInterval(bucle);  
21     } else {  
22         medio.play();  
23         reproducir.value = "||";  
24         bucle = setInterval(estado, 1000);  
25     }  
26 }
```

- La función `presionar()` se ejecuta cuando el usuario hace clic en el botón ">" (reproducir).
 - Este botón tiene dos propósitos: muestra el carácter ">" para reproducir el vídeo o "||" para pausarlo, de acuerdo al estado actual.
 - Para determinar esta condición, el código detecta el estado del medio leyendo las propiedades **paused** y **ended**. Esto lo realiza la instrucción **if** en la primera línea de la función. Si los valores de las propiedades **paused** y **ended** son **false**, significa que el vídeo se está reproduciendo y, por lo tanto, se ejecuta el método **pause()** para pausarlo y el título del botón cambia a ">".
-
- En esta oportunidad, aplicamos el operador **!** (NO lógico) a cada propiedad para lograr el propósito. Si las propiedades devuelven **false**, el operador cambia el valor a **true**.
 - La instrucción **if** se debe leer como "si el medio *no* se ha pausado y el medio *no* ha finalizado, entonces hacer esto "
 - Si el vídeo se ha pausado o se ha terminado de reproducir, la condición es **false** y el método **play()** se ejecuta para comenzar a reproducir o continuar reproduciendo el vídeo.
 - En este caso, también realizamos una tarea importante que es la de iniciar la ejecución de la función **estado()** cada segundo con el método **setInterval()** del objeto **Window** para actualizar la barra de progreso.

reproductor.js

```
27 function estado() {
28     if (!medio.ended) {
29         var largo = parseInt(medio.currentTime * maximo / medio.duration);
30         progreso.style.width = largo + "px";
31     } else {
32         progreso.style.width = "0px";
33         reproducir.value = ">";
34         clearInterval(bucle);
35     }
36 }
```

- La función `estado()` se ejecuta cada segundo mientras se reproduce el vídeo. La instrucción `if` controla el estado del vídeo. Si la propiedad **ended** devuelve **false**, se calcula la longitud que debería tener la barra de progreso en píxeles y declara el nuevo tamaño para el elemento `<div>` que la representa. Pero si el valor de la propiedad **ended** es **true** (significa que el vídeo ha finalizado), declara el tamaño de la barra de progreso nuevamente a 0 píxeles, cambiamos el texto del botón a ">" (reproducir) y cancela el bucle con el método `clearInterval()`. Después de esto, la función **estado()** ya no se ejecuta.
- Para calcular el valor actual, necesitamos el valor de la propiedad **currentTime** para saber qué parte del vídeo se está reproduciendo, el valor de la propiedad **duration** para saber la duración del vídeo y el valor de la variable **maximo** para obtener el tamaño máximo permitido para la barra de progreso.
- La fórmula es una regla de tres simple. Se multiplica el tiempo actual por el tamaño máximo y dividir el resultado por la duración (**tiempo-actual × maximo / duración**). El resultado es el nuevo tamaño en píxeles del elemento `<div>` que representa la barra de progreso.
- La función que responde al evento **click** del elemento reproducir (**el botón de reproducir**) ya se ha creado, por lo que es hora de hacer lo mismo para el evento click de la barra de progreso. A este método lo llamamos **mover()**

reproductor.js

```
37 function mover(evento) {  
38     if (!medio.paused && !medio.ended) {  
39         var ratonX = evento.offsetX - 2;  
40         if (ratonX < 0) {  
41             ratonX = 0;  
42         } else if (ratonX > maximo) {  
43             ratonX = maximo;  
44         }  
45         var tiempo = ratonX * medio.duration / maximo;  
46         medio.currentTime = tiempo;  
47         progreso.style.width = ratonX + "px";  
48     }  
49 }
```

- En la función `iniciar()`, se agrego un listener al elemento barra para el evento **click** con la intención de responder cada vez que el usuario quiere comenzar a reproducir el vídeo desde una nueva posición.
 - Cuando este evento se desencadena, se ejecuta la función **mover()**. Esta función comienza con una instrucción **if**, como las anteriores funciones, pero esta vez el objetivo es llevar a cabo la acción solo cuando el vídeo se está reproduciendo. Si las propiedades **paused** y **ended** devuelven **false**, significa que el vídeo se está reproduciendo y que podemos ejecutar el código.
-
- Para calcular el tiempo en el que se debe seguir reproduciendo el vídeo, obtenemos la posición del ratón desde la propiedad **offsetX**, calculamos el tamaño de la barra de progreso considerando el relleno del elemento barra y luego convertimos este tamaño en segundos para comenzar a reproducir el vídeo desde la nueva ubicación. El valor en segundos que representa la posición del ratón en la línea de tiempo se calcula con la fórmula **ratonX × medio.duration / maximo**. Una vez que se obtiene este valor, hay que asignarlo a la propiedad **currentTime** para comenzar a reproducir el vídeo en esa posición. Al final, la posición del ratón se asigna a la propiedad **width** del elemento progreso para reflejar el nuevo estado del vídeo en la pantalla.
 - Además de estas funciones, necesitamos dos más para controlar el audio del medio. La primera es la función **sonido()**, asignada como listener del evento **click** del botón Silencio.

reproductor.js

```
50 function sonido() {  
51     if (silenciar.value == "Silencio") {  
52         medio.muted = true;  
53         silenciar.value = "Sonido";  
54     } else {  
55         medio.muted = false;  
56         silenciar.value = "Silencio";  
57     }  
58 }
```

- La función **sonido()** activa o desactiva el sonido del medio dependiendo del valor del atributo **value** del botón Silencio.
- El botón muestra diferentes textos de acuerdo a la situación. Si el valor actual es "Silencio", el sonido se desactiva y el título del botón cambia a "Sonido". En caso contrario, el sonido se activa y el título del botón cambia a "Silencio".
- Cuando el sonido está activo, el volumen se puede controlar a través del elemento **volumen**, localizado al final de la barra de progreso.
- El elemento desencadena el evento **change** cada vez que se modifica su valor (cada vez que se desplaza el control). A la función que responde al evento la llamamos **nivel()**.

reproductor.js

```
59 function nivel() {  
60     medio.volume = volumen.value;  
61 }
```

- La función **nivel()** asigna el valor del atributo `value` del elemento `<input>` que representa el control a la propiedad **volume** del medio. Lo único que tenemos que recordar es que esta propiedad acepta valores entre 0.0 y 1.0.
- Los números fuera de este rango devolverán un error.

```
62 window.addEventListener("load", iniciar);
```

Lo último que queda es responder al evento `load` para iniciar la aplicación cuando se descarga el documento.

Subtítulos

- Los subtítulos son texto que se muestra sobre el reproductor mientras se reproduce el vídeo.
- Este sistema se ha utilizado durante décadas en televisión y en diferentes medios de distribución de vídeo, pero hasta este momento no había sido fácil incluirlo en la Web. HTML ofrece el siguiente elemento para simplificar su implementación.

<track>—Este elemento agrega subtítulos a un vídeo.

- El elemento tiene que incluirse dentro de un elemento **<video>** o **<audio>**. Para especificar la fuente, el tipo y el modo en el que se mostrarán los subtítulos en la pantalla

Atributos del elemento `<track>`

- **src**—Este atributo declara la URL del archivo que contiene el texto de los subtítulos. El formato de este archivo puede ser cualquiera de los que admiten los navegadores, pero la especificación declara al formato WebVTT como el oficial para este elemento.
- **srclang**—Este atributo declara el idioma del texto. Trabaja con los mismos valores que el atributo `lang` del elemento `<html>`
- **default**—Este atributo declara la pista (*track*) que queremos incluir por defecto. Si solo se facilita un elemento `<track>`, este atributo se puede usar para activar los subtítulos.
- **label**—Este atributo facilita un título para la pista. Si se incluyen varios elementos `<track>`, este atributo ayuda a los usuarios a encontrar el correcto.
- **kind**—Este atributo declara el tipo de contenido asignado a una pista. Los valores disponibles son **subtitles** (para subtítulos), **captions** (para subtítulos que representan sonido), **descriptions** (destinado a sintetizado de audio), **chapters** (para navegación entre capítulos) y **metadata** (para información adicional que no se muestra en la pantalla). El valor por defecto es **subtitles** (subtítulos).

```

1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4  |   <meta charset="utf-8">
5  |   <title>Subtítulos</title>
6  </head>
7  <body>
8  |   <section>
9  | |   <video width="720" height="400" controls>
10 | | |   <source src="Post-PC.mp4">
11 | | |   <source src="Post-PC.ogv">
12 | | |   <track src="subtitulos.vtt" srclang="es" default>
13 | |   </video>
14 |   </section>
15 </body>
16 </html>

```

- En este ejemplo se declara un solo elemento **<track>**. El idioma de la fuente se ha declarado como Español, y se ha incluido el atributo **default** para declarar esta pista como la pista por defecto y de este modo activar los subtítulos.
- La fuente de este elemento se ha declarado como un archivo en formato WebVTT (subtitulos.vtt).
- **WebVTT** son las siglas del nombre **Web Video Text Tracks**, un formato estándar para subtítulos.
- Los archivos de este formato son simplemente texto con una estructura especial, tal como ilustra el siguiente ejemplo:

```

WEBVTT
00:02.000 --> 00:07.000
Bienvenido
al elemento <track>
00:10.000 --> 00:15.000
Este es un ejemplo simple.
00:17.000 --> 00:22.000
Varias pistas pueden ser usadas simultáneamente
00:22.000 --> 00:25.000
para ofrecer textos en diferentes lenguajes.
00:27.000 --> 00:30.000
Hasta luego!

```

```
WEBVTT
00:02.000 --> 00:07.000
<i>Bienvenido</i>
al elemento <track>!
00:10.000 --> 00:15.000
<v Roberto>Este es un ejemplo <c.titulos>simple</c>.
00:17.000 --> 00:22.000
<v Martin>Se pueden usar varias pistas simultáneamente
00:22.000 --> 00:25.000
<v Martin>para ofrecer textos en diferentes lenguajes.
00:27.000 --> 00:30.000
<b>Hasta luego!</b>
```

```
::cue(.titulos) {
color: #990000;
}
```

- Las *cues* (las líneas de texto en un archivo de subtítulos) pueden incluir las etiquetas especiales , <i>, <u>, <v> y <c>.
- Las tres primeras son para otorgar énfasis, como en HTML, mientras que la etiqueta <v> declara a quién pertenece el texto (quien habla) y la etiqueta <c> nos permite asignar estilos usando CSS.
- El formato WebVTT usa un seudoelemento para referenciar las *cues*. Este seudoelemento, llamado `::cue`, puede recibir un selector de una clase entre paréntesis. En el ejemplo, la clase se ha llamado *titulos* (<c.titulos>), por lo que el selector CSS se debe escribir como `::cue(.titulos)`, como muestra el ejemplo.

El formato WebVTT ofrece los siguientes parámetros y valores:

- **align**—Este parámetro alinea la **cue** en relación al centro del espacio que cubre el medio. Los valores disponibles son **start**, **middle**, y **end**.
- **vertical**—Este parámetro cambia la orientación a vertical y ordena la **cue** de acuerdo a dos valores: **rl** (derecha a izquierda) o **lr** (izquierda a derecha).
- **position**—Este parámetro declara la posición de la **cue** en columnas. El valor se puede expresar como un porcentaje o como un número de 0 a 9. La posición se declara de acuerdo a la orientación.
- **line**—Este parámetro declara la posición de la **cue** en filas. El valor se puede expresar en porcentaje o como un número de 0 a 9. En una orientación horizontal, la posición que se declara es vertical, y viceversa. Los números positivos declaran la posición desde un lado y los números negativos desde el otro, dependiendo de la orientación.
- **size**—Este valor declara el tamaño de la **cue**. El valor se puede declarar en porcentaje y se determina a partir del ancho del medio.

- Estos parámetros y sus correspondientes valores se declaran al final de la ***cue*** separados por dos puntos. Se pueden hacer varias declaraciones para la misma *cue*, como muestra el siguiente ejemplo:

```
WEBVTT
00:02.000 --> 00:07.000 align:start
position:5%
<i>Bienvenido</i>
al elemento &lt;track&gt;!
```

API TextTrack

- La API TextTrack se definió para ofrecer acceso desde JavaScript al contenido de las pistas usadas como subtítulos. La API incluye un objeto llamado TextTrack para devolver esta información. Existen dos maneras de obtener este objeto: desde el elemento de medios o desde el elemento <track>.
- **textTracks**—Esta propiedad contiene un array con los objetos **TextTrack** de cada pista del medio. Los objetos **TextTrack** se almacenan en el array en orden secuencial.
- **track**—Esta propiedad devuelve el objeto **TextTrack** de la pista especificada.
 - Si el medio (vídeo o audio) contiene varios elementos <track>, puede resultar más fácil encontrar la pista que buscamos accediendo al array **textTracks**.

```

1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>Trabajando con Pistas</title>
6  <script>
7      function iniciar() {
8          var video = document.getElementById("medio");
9          var pista1 = video.textTracks[0];
10         var pista = document.getElementById("pista");
11         var pista2 = pista.track;
12         console.log(pista1);
13         console.log(pista2);
14     }
15     window.addEventListener("load", iniciar);
16 </script>
17 </head>
18 <body>
19     <section>
20         <video id="medio" width="720" height="400" controls>
21             <source src="trailer.mp4">
22             <source src="trailer.ogg">
23             <track id="pista" label="Subtítulos en Español"
24                 src="subtitulos.vtt" srclang="es" default>
25         </video>
26     </section>
27 </body>
28 </html>

```

El ejemplo demuestra cómo acceder al objeto **TextTrack** usando ambas propiedades.

La función **iniciar()** crea una referencia al elemento **<video>** para obtener el objeto **TextTrack** accediendo al array **textTracks** con el índice 0 y luego se obtiene nuevamente el mismo objeto desde el elemento **<track>** usando la propiedad **track**.

Finalmente, el contenido de ambas variables se imprime en la consola.

Leyendo pistas

Una vez que obtenemos el objeto **TextTrack** de la pista con la que queremos trabajar, podemos acceder a sus propiedades.

- **kind**—Esta propiedad devuelve el tipo de pista, tal como se ha especificado mediante el atributo **kind** del elemento **<track>** (subtitles, captions, descriptions, chapters o metadata).
- **label**—Esta propiedad devuelve la etiqueta de la pista, tal como se ha especificado con el atributo **label** del elemento **<track>**.
- **language**—Esta propiedad devuelve el idioma de la pista, tal como se ha especificado mediante el atributo **srclang** del elemento **<track>**.
- **mode**—Esta propiedad devuelve o declara el modo de la pista. Los valores disponibles son **disabled** (desactivada), **hidden** (oculta) y **showing** (mostrar). Se puede usar para intercambiar pistas.
- **cues**—Esta propiedad es un array que contiene todas las **cues** de la pista.
- **activeCues**—Esta propiedad devuelve las **cues** que actualmente se están mostrando en pantalla (la anterior, la actual y la siguiente).

A partir de las propiedades del objeto TextTrack podemos acceder a toda la información almacenada en la pista.

```

1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>Trabajando con Pistas</title>
6  <style>
7      #reproductor, #info {
8          float: left;
9      }
10 </style>
11 <script>
12     function iniciar() {
13         var info = document.getElementById("info");
14         var pista = document.getElementById("pista");
15         var obj = pista.track;
16         var lista = "";
17         lista += "<br>Tipo: " + obj.kind;
18         lista += "<br>Etiqueta: " + obj.label;
19         lista += "<br>Idioma: " + obj.language;
20         info.innerHTML = lista;
21     }
22     window.addEventListener("load", iniciar);
23 </script>
24 </head>
25 <body>
26     <section id="reproductor">
27         <video id="medio" width="720" height="400" controls>
28             <source src="trailer.mp4">
29             <source src="trailer.ogg">
30             <track id="pista" label="Subtítulos en Español"
31                 src="subtitulos.vtt" srclang="es" default>
32         </video>
33     </section>
34     <aside id="info"></aside>
35 </body>
36 </html>

```

- El ejemplo incluye estilos para las dos columnas creadas por los elementos **<section>** y **<aside>**, y el código JavaScript para obtener y mostrar los datos del elemento **<track>**.
- Esta vez, obtenemos el objeto **TextTrack** desde la propiedad **track** y lo almacenamos en la variable **obj**.
- Desde esta variable, leemos los valores de las propiedades del objeto y generamos el texto para mostrarlos en pantalla.

Leyendo *cues*

- Además de las propiedades aplicadas en el último ejemplo, también contamos con una propiedad importante llamada *cues*.
- Esta propiedad contiene un **array** con objetos **TextTrackCue** que representan cada *cue* de la pista.

Propiedades de TextTrackCue

- **text**—Esta propiedad devuelve el texto de la *cue*.
- **startTime**—Esta propiedad devuelve el tiempo de inicio de la *cue* en segundos.
- **endTime**—Esta propiedad devuelve el tiempo de finalización de la *cue* en segundos.
- **vertical**—Esta propiedad devuelve el valor del parámetro **vertical**. Si el parámetro no se ha definido, el valor que devuelve es una cadena de caracteres vacía.
- **line**—Esta propiedad devuelve el valor del parámetro **line**. Si el parámetro no se ha definido, el valor se devuelve por defecto.
- **position**—Esta propiedad devuelve el valor del parámetro **position**. Si el parámetro no se ha especificado, el valor se devuelve por defecto.
- **size**—Esta propiedad devuelve el valor del parámetro **size**. Si el parámetro no se ha definido, el valor se devuelve por defecto.
- **align**—Esta propiedad devuelve el valor del parámetro **align**. Si el parámetro no se ha definido, el valor se devuelve por defecto.

```

1  <script>
2      function iniciar() {
3          var info = document.getElementById("info");
4          var pista = document.getElementById("pista");
5          var obj = pista.track;
6          var lineas = obj.cues;
7          var lista = "";
8          for (var f = 0; f < lineas.length; f++) {
9              lista += lineas[f].text + "<br>";
10         }
11         info.innerHTML = lista;
12     }
13     window.addEventListener("load", iniciar);
14 </script>

```

- La nueva función **iniciar()** accede a cada *cue* usando un bucle for.
- Dentro del bucle, los valores del array se agregan a la variable lista junto con un elemento
 para mostrar las *cues* una por línea, y luego todo el texto se inserta dentro del elemento <aside> para mostrarse en pantalla.
- Los objetos TextTrackCue incluyen propiedades con la información de cada *cue*. En este ejemplo, mostramos el contenido de la propiedad text.

```

1  <script>
2      function iniciar() {
3          var info = document.getElementById("info");
4          var pista = document.getElementById("pista");
5          var obj = pista.track;
6          var lineas = obj.cues;
7          var lista = "";
8          for (var f = 0; f < lineas.length; f++) {
9              var linea = lineas[f];
10             lista += linea.startTime + " - ";
11             lista += linea.text + "<br>";
12         }
13         info.innerHTML = lista;
14     }
15     window.addEventListener("load", iniciar);
16 </script>

```

- El siguiente código actualiza el ejemplo anterior para agregar los tiempos de inicio de cada *cue*.

Agregando pistas

- El objeto `TextTrack` que representa una pista no solo tiene propiedades, sino también métodos con los que podemos crear nuevas pistas y *cues* desde JavaScript.
- **`addTextTrack(tipo, etiqueta, idioma)`**—Este método crea una nueva pista para el medio y devuelve el objeto `TextTrack` correspondiente. Los atributos son los valores de los atributos para la pista (solo **`tipo`** es obligatorio).
- **`addCue(objeto)`**—Este método agrega una nueva *cue* a la pista. El atributo **`objeto`** es un objeto `TextTrackCue` que devuelve el constructor `VTT Cue()`.
- **`removeCue(objeto)`**—Este método elimina una *cue* de la pista. El atributo **`objeto`** es un objeto `TextTrackCue` que devuelve el objeto `TextTrack`.
- Para agregar *cues* a la pista, tenemos que proveer un objeto `TextTrackCue`. La API incluye un constructor para crear este objeto.
- **`VTT Cue(inicio, finalización, texto)`**—Este constructor devuelve un objeto `TextTrackCue` para usar con el método `addCue()`. Los atributos representan los datos para la *cue*.

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="utf-8">
5   <title>Trabajando con Pistas</title>
6 <script>
7 function iniciar(){
8   var linea;
9   var lineas = [
10    { start: 2.000, end: 7.000, text: "Bienvenido"},
11    { start: 10.000, end: 15.000, text: "Este es un ejemplo"},
12    { start: 15.001, end: 20.000, text: "de como agregar"},
13    { start: 20.001, end: 25.000, text: "una nueva pista."},
14    { start: 27.000, end: 30.000, text: "Hasta Luego!"},
15   ];
16   var video = document.getElementById("medio");
17   var nuevapista = video.addTextTrack("subtitles");
18   nuevapista.mode = "showing";
19   for (var f = 0; f < lineas.length; f++) {
20     linea = new VTTCue(lineas[f].start, lineas[f].end, lineas[f].text);
21     nuevapista.addCue(linea);
22   }
23   video.play();
24 }
25 window.addEventListener("load", iniciar);
26 </script>
27 </head>
28 <body>
29   <section>
30     <video id="medio" width="720" height="400" controls>
31       <source src="trailer.mp4">
32       <source src="trailer.ogg">
33     </video>
34   </section>
35 </body>
36 </html>

```

- En este ejemplo, comenzamos la función **iniciar()** definiendo el array *cues* con cinco *cues* para nuestra pista. Las *cues* se declaran como elementos del array. Cada *cue* es un objeto con las propiedades **start**, **end** y **text**. Los valores de los tiempos de inicio y finalización no usan la sintaxis del archivo WebVTT; en cambio, se tienen que declarar en segundos con números decimales.
- Las *cues* se pueden agregar a una pista existente o a una nueva. En nuestro ejemplo, hemos creado una nueva pista de tipo subtitles usando el método **addTextTrack()**.
- También tenemos que declarar el modo (mode) de esta pista como **showing**, para pedirle al navegador que muestre la pista en la pantalla. Cuando la pista está lista, todas las *cues* del array *cues* se convierten en objetos **TextTrackCue** y se agregan a la pista con el método **addCue()**.